Institutional Capability Lineages: A Registry-Centered Reference Architecture for Governed and Evolving AI

Canonical Capability Contracts, Contextual Projection, and Evidence-Governed Institutional Evolution

Mariano Garralda-Barrio*
Independent Researcher
Lleida, Spain
mariano.garralda.r@gmail.com

July 22, 2026

Abstract

AI systems increasingly retrieve knowledge, maintain memory, assemble context, load skills, and act through tools. Organizations, however, must keep recurring capabilities executable across changing people, agents, models, workflows, and infrastructures. This creates an institutional continuity problem: which capability owns the required knowledge, which contract is authoritative, how situated execution views are derived, and how evidence may revise future state. This paper introduces *Institutional Capability Lineages* (ICLA), a registry-centered reference architecture that makes capability the stable unit of knowledge responsibility. The *Institutional Capability Registry* maintains navigable identity, ownership, relations, lifecycle, and active-contract pointers while governed source knowledge remains distributed. Immutable, consumer-independent *Capability Knowledge Contracts* (CKCs) define canonical capability knowledge and evaluation commitments; admitted contextual assemblies instantiate bounded operational mandates for situated *Capability Execution Environments* (CEEs) used by humans, agents, workflows, systems, or hybrids. CEEs reason and iterate with local working state, while contract-selected outputs return as governed evidence from which adjudication may activate a successor CKC or initiate capability crystallization. A lineage connects capability identity to exact contracts, assemblies, executions, evidence, decisions, and lifecycle changes. ICLA thereby makes capability contracts, autonomous contextual execution, and institutional evolution jointly accountable. We define its information model, conformance invariants, interaction contracts, reference artifacts, and comparative evaluation protocol.

Keywords institutional capability lineages · capability execution environments · institutional capability registry · Capability Knowledge Contract · knowledge-centric AI · organizational knowledge · context engineering · knowledge governance · institutional evolution

1 Introduction

AI systems increasingly combine retrieval-augmented generation, agent memory, context assembly, operational skills, and tool-mediated action [1–4]. These mechanisms improve task-level access and adaptation. The institutional problem begins earlier and lasts longer: organizations must preserve the knowledge that keeps recurring capabilities executable across changes in people, models, agents, workflows, repositories, and infrastructure.

*Independent Researcher / Investigador Independiente.

Capabilities commonly outlive the mechanisms used to perform them. A bank may replace its AI platform while retaining credit-risk assessment; a software organization may change its incident tooling while retaining incident diagnosis; and a regulated enterprise may adopt new automation while retaining compliance validation. Organizational-memory research explains how knowledge persists across people, routines, artifacts, and repositories [5–7]. Enterprise architecture provides stable capability identities that survive changes in processes and technology [8, 9]. What remains missing is the accountable connection between the two.

We use *knowledge-for-capability* for the knowledge an institution must maintain to perform a recurring capability responsibly. Functionally, it may combine semantic knowledge—concepts, policies, and approved interpretations—procedural knowledge—methods, controls, and workflows—and episodic knowledge—prior executions, decisions, exceptions, and evidence. These are overlapping analytical modalities, not storage layers or claims of cognitive equivalence [10, 11]. The architectural question is which sources and commitments belong to a capability, who governs them, how they are projected, and under what evidence they may change.

Execution is increasingly heterogeneous. A capability instance may be interpreted by a domain expert, planned by an AI agent, carried by a workflow, verified by software, and approved by an accountable human. Multi-agent and human–agent systems reinforce this pattern through role specialization, tool use, and varied coordination structures [12–15]. We call the situated boundary in which such work occurs a *Capability Execution Environment* (CEE). A CEE may be a human or team, an agent, a workflow, a software service, or a hybrid. Multiple CEEs performing the same or related capabilities distribute cognition, local state, and knowledge production across the organization [16, 17].

Queries, prompts, memories, retrieved passages, skills, and tool calls provide task-time behavior. Persistent capability ownership, a canonical contract for knowledge and evaluation, comparable measures across CEEs, and promotion authority require an institutional layer around them. Catalogs and provenance models add discovery and lineage [18, 19]; architecture and risk standards add accountability and controlled change [20, 21]. The unresolved architectural responsibility is to connect these mechanisms around a stable capability and preserve that connection over time.

Consider an organization introducing OAuth 2.1 across several services. Architects interpret policy, coding agents modify services, test systems collect compatibility evidence, and security reviewers decide acceptance. These CEEs require different working views and may produce complementary knowledge. The organization still needs one stable capability address, one authoritative contract for its knowledge and evaluation commitments, and one governed path by which evidence can change that contract.

Distributed execution requires a canonical capability contract and a governed return path.

Reasoning and sources remain distributed; capability responsibility, authority, and change become explicit.

Knowledge-centric AI names the broader research program in which systems operate through explicit, governed, reusable knowledge structures rather than treating context as an incidental prompt artifact [22]. This paper proposes *Institutional Capability Lineages* (ICLA), a reference architecture within that program.

ICLA makes capability the primary navigation and accountability object. An *institutional capability* is a recurring, outcome-oriented responsibility with a stable identity, accountable owner, and governed lifecycle across consumers and implementations. The *Institutional Capability Registry* records that identity, its relations and lifecycle, and the pointer to the active *Capability Knowledge Contract* (CKC). Governed source knowledge remains distributed and is reached through explicit bindings.

A CKC is the consumer-independent contract for knowledge-for-capability:

> *A Capability Knowledge Contract is a governed, versioned, and consumer-independent specification of the knowledge scope, operational relations, obligations, authorities, evidence requirements, evaluation contract, source bindings, and projection rules required to perform a recurring institutional capability under declared conditions.*

Operational intents resolve against active CKCs. Admissible resolutions produce contextual assemblies for identified CEEs, risks, assurance levels, and budgets. Admission and materialization establish a bounded operational mandate recorded by the assembly. The CEE may organize the delivery, combine it with local state, and reason, plan, invoke tools, or iterate without step-wise ICLA interaction. Re-resolution occurs only when the mandate becomes insufficient or invalid; selected outputs return under the evidence contract. Across repeated intents, governed succession may refine an existing capability contract, while recurrent patterns

may support a crystallization proposal for a new capability and initial CKC. Neither path is automatic: CEE-produced knowledge may influence reusable institutional state only through qualification and explicit governance. An *institutional capability lineage* is the auditable continuity connecting capability identity to exact CKC versions, assemblies, executions, evidence, decisions, and lifecycle changes.

This Registry-addressable lineage makes stable capability identity, canonical contracts, contextual projection, comparable evidence, and governed institutional evolution jointly accountable. The lineage—not the Registry or CKC in isolation—is the architectural unit that preserves continuity across changing contracts, CEEs, executions, evidence, and decisions. ICLA separates distributed execution and knowledge production from institutional authority: humans, agents, workflows, and systems may reason and produce knowledge locally, while capability identity, canonical contracts, evidence admission, and institutional evolution remain governed, traceable, and auditable.

The paper addresses the following question:

> *How can a registry-centered architecture preserve institutional capability continuity while supporting canonical capability contracts, contextual execution, comparable evidence, and governed evolution across heterogeneous execution environments?*

The paper's primary architectural contribution is a registry-centered capability and lineage model that connects stable responsibility and navigable metadata to distributed sources, heterogeneous CEEs, and immutable execution traces. It is realized through three supporting contributions:

- **The Capability Knowledge Contract and projection boundary** separating the canonical, consumer-independent contract from contextual assemblies and local execution state.
- **An evidence-governed evolution model** that records explicit successor deltas, preserves historical states, and distinguishes CKC succession from capability crystallization.
- **A conformance and evaluation framework** comprising invariants, profiles, logical interfaces, machine-checkable schemas, conforming reference traces, a deterministic Reference Implementation, executable tests, and falsifiable comparative claims.

A versioned companion artifact set operationalizes this framework through eight machine-checkable object schemas, linked reference traces, a deterministic Reference Implementation, and executable conformance tests; Section 10.2 defines its scope and evidentiary role.

The architecture is independent of database, model, CEE coordination topology, transport, and knowledge format. Section 2 positions the proposal against institutional knowledge, enterprise capabilities, distributed execution, runtime context, provenance, and governance. Section 3 derives the architecture from the resulting gap. Sections 4–8 define the information model, Registry, contracts, execution boundary, interaction architecture, and governed evolution. Sections 9–12 specify conformance, implementation, a worked trace, and evaluation; the final sections state scope and conclusions.

2 Foundations and Related Work

This section positions ICLA across institutional knowledge, enterprise capability, distributed human–agent execution, runtime context, provenance, and governance research, then isolates the architectural gap.

2.1 Institutional knowledge and capability foundations

Organizational-memory and knowledge-management research treats knowledge as distributed across people, routines, artifacts, and repositories, and explains its retention, transfer, and governance [5, 6, 23, 7, 17, 24]. Enterprise architecture contributes stable capability identities and relations that survive changes in structures, processes, and technologies [8, 25, 9, 26]. ICLA treats the memory modalities as overlapping organizational knowledge roles and joins them to stable capability identity through a machine-operational contract.

2.2 Distributed capability execution

Multi-agent systems distribute problem solving across participants with distinct roles, knowledge, tools, and interaction structures. Contemporary LLM-based systems support role specialization and centrally orchestrated, peer-to-peer, hierarchical, and decentralized coordination [12–14]. Human–agent systems add

human information, feedback, control, and decision authority [15]. Together these works motivate distributed capability execution across heterogeneous CEEs. ICLA adds the persistent institutional layer that binds each admitted execution to capability ownership, canonical contracts, comparable evidence, and promotion authority.

2.3 Runtime knowledge and context systems

Data catalogs, knowledge graphs, and provenance models support discovery, relations, governance, lineage, and impact analysis [18, 19]. Retrieval-augmented generation, context engineering, and agent-memory systems optimize runtime selection, adaptation, and continuity [1, 3, 27, 2, 28, 29]. Knowledge Activation and agent-skill registries package operational knowledge for reuse [30, 4]. Their primary units remain data assets, queries, contexts, memories, or portable skills rather than institutionally owned recurring responsibilities. These mechanisms may operate inside assembly or CEE execution; ICLA remains outside the executor's step-wise loop.

2.4 Governance, epistemic control, and interoperability

Epistemic infrastructures and context orchestration add typed commitments, contradiction handling, permissions, freshness, reconciliation, and delivery controls [31, 32]. Architecture-description and AI risk-management standards require explicit concerns, accountable authority, traceability, and controlled change [20, 21]. The Open Knowledge Format (OKF) and Model Context Protocol (MCP) provide interoperable representations and operations [33, 34]. ICLA places these mechanisms within capability ownership, contract activation, evidence admission, and institutional lifecycle.

2.5 Architectural gap

Across these research areas, five responsibilities remain distributed among separate abstractions: stable identity for recurring responsibility; a canonical, consumer-independent knowledge and evaluation contract; intent-specific projections; an explicit boundary for situated human–machine execution; and an adjudication path that can evolve a contract or form a capability while preserving history. ICLA makes their connection the primary architectural object. Table 1 summarizes the positioning.

Table 1: Adjacent approaches and the responsibilities integrated by ICLA.

Approach	**Primary unit**	**Main strength**	**ICLA responsibility added**
Organizational memory / knowledge management	Retained knowledge, routines, practices	Institutional retention and learning	Capability contract and projection
Business capability maps	Capability identity and hierarchy	Stable enterprise navigation and planning	Canonical contract and evidence loop
Multi-agent / human–agent systems	Agent collective or situated CEE	Distributed reasoning, specialization, and human participation	Capability ownership and evidence admission
Catalogs / provenance / graphs	Metadata entities and relations	Discovery, governance, lineage, impact analysis	Capability identity and active authority
RAG / context / memory / skills	Query, context, memory, operational unit	Runtime selection, adaptation, continuity, and reuse	Canonical boundary and evaluation
Epistemic and context orchestration	Typed knowledge or context resource	Commitments, permissions, freshness, reconciliation, delivery	Lifecycle and promotion authority

This synthesis motivates the architectural requirements derived next: distributed knowledge retains source authority; institutional responsibility remains addressable through canonical contracts; CEEs preserve situated autonomy and explicit execution identity; projections remain contextual and reproducible; and returned evidence enters a controlled, incremental evolution path.

3 Architectural Requirements and Component Derivation

We translate the literature gap into explicit responsibilities and protected boundaries, following architecture-description principles that separate concerns from implementations [20]. Figure 1 presents the resulting architecture at its simplest; Table 2 traces each component to the boundary it protects.

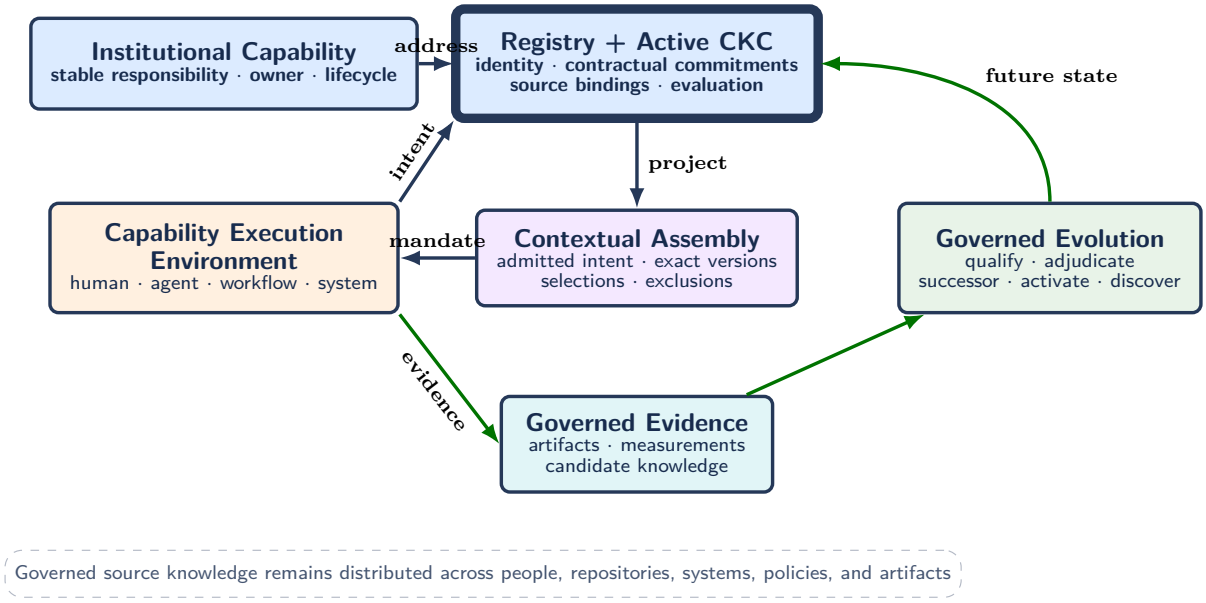

Figure 1: ICLA in one view. Registry and active CKC resolve a stable capability; an admitted assembly carries a bounded operational mandate into a CEE with local working state; selected evidence returns to governed evolution. Connected records form the institutional capability lineage.

Table 2: Derivation of ICLA components from architectural requirements.

Observed condition	**Required responsibility**	**ICLA object and protected boundary**
Distributed sources and policies change independently	Expose versions, bindings, and change events without moving source authority	Organizational Memory; distributed source governance
Responsibilities outlive CEE configurations and implementations	Preserve stable identity, ownership, relations, lifecycle, and active authority	Registry; institutional continuity
Capability metadata cannot express the required commitment	Define the canonical contract for knowledge, evaluation, evidence, authority, and projection	CKC; explicit contract semantics
Heterogeneous execution requires situated views and delivery forms	Establish a bounded operational mandate while preserving local autonomy and traceability	CEE, Assembly, and Materialization; contract/projection boundary
Sources and executions produce candidate institutional inputs	Qualify, adjudicate, and activate change without direct canonical mutation	Evidence Gateway and Governance; governed evolution
Repeated work may reveal an unmodeled responsibility	Review boundary, ownership, value, and overlap before assigning identity	Crystallization; governed capability formation

The derived components form two connected paths: an operational path from a CEE intent to a situated projection, and an institutional path from source change or execution evidence to governed future state. The

Registry and CKC provide compact metadata, contractual authority, and governed pointers to distributed source data. The next section formalizes the objects and transitions shared by both paths.

4 Core Information and Transition Model

The model separates canonical institutional state, intent-specific execution state, and governed transition records. Canonical state comprises distributed governed sources, stable capability identities, and immutable CKC versions; execution state comprises resolutions, assemblies, materializations, and evidence. Transitions may change current authority without altering objects referenced by earlier executions.

4.1 Organizational memory

Let $\mathcal{M}_t$ denote versioned organizational memory at time t: documents, code, policies, decisions, data, approved interpretations, observations, and evidence across repositories. Its governed contents may play overlapping semantic, procedural, or episodic roles. These roles characterize knowledge function rather than storage location, and one source may support several roles. Organizational Memory exposes versions, bindings, retention, and change events while source owners retain content authority. Source changes initiate impact analysis over affected capabilities and CKCs.

4.2 Institutional Capability Registry

Equation (1) models the registry as a time-indexed attributed graph:

$$\Gamma_t = \langle \mathcal{C}_t, \mathcal{E}_t, \mu_t, \nu_t \rangle, \tag{1}$$

where $\mathcal{C}_t$ is the set of institutional capability identities, $\mathcal{E}_t$ is the set of typed relations, μ_t maps capabilities to metadata, and ν_t maps each capability to its lifecycle and active CKC version.

Relations such as *depends on*, *specializes*, *replaces*, and *shares knowledge with* support resolution, impact analysis, and lifecycle management. For each $c \in \mathcal{C}_t$, $\mu_t(c)$ includes identity, name, outcome, owner, domain, lifecycle, risk, maturity, and applicable policy. Authorized actors govern this metadata and the CKC pointers; resolvers filter it by intent, CEE, conditions, risk, and policy before following bindings to source payloads. The Registry therefore indexes distributed knowledge rather than duplicating it (Figure 2).

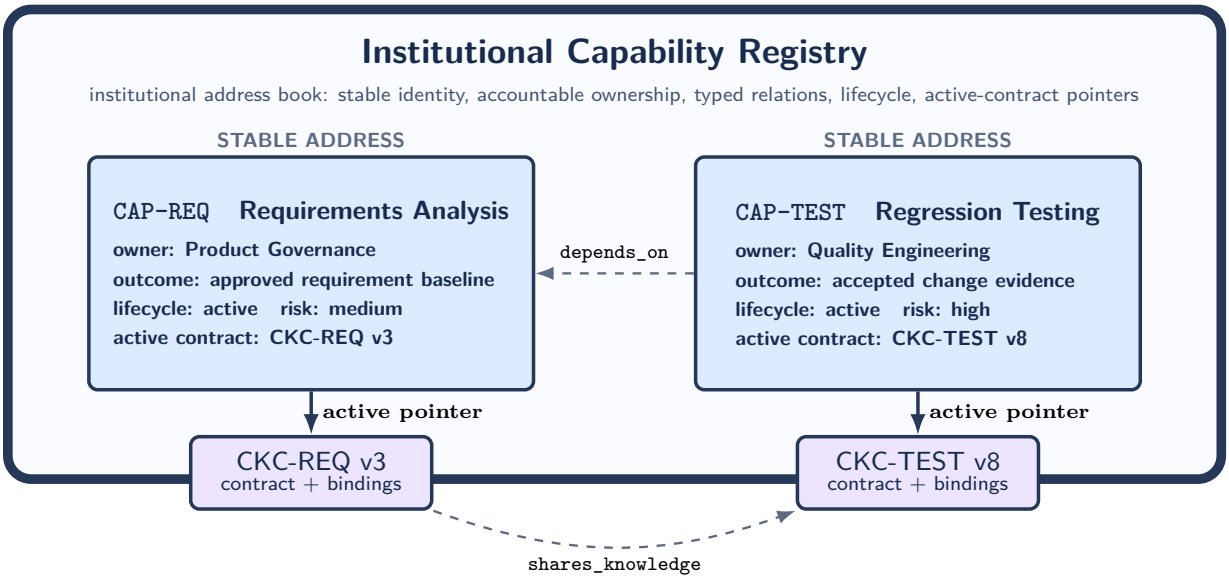

Figure 2: The Registry as an institutional address book. Capability identities and relations remain stable while governed pointers select the active capability contracts.

4.3 Capability Knowledge Contract

Equation (2) defines CKC version v for capability c:

$$K_c^v = \langle S_c^v, R_c^v, E_c^v, Q_c^v, G_c^v, A_c^v \rangle, \tag{2}$$

Here S defines knowledge scope and source bindings; R requirements, obligations, and operational relations; E evidence schemas; Q evaluation metrics and interpretation; G governance and authorities; and A projection and assembly rules. The scope and bindings in S may cover semantic commitments, procedural guidance, and episodic precedents; A governs which authorized elements enter an assembly for a given intent and CEE. These are logical source roles, not mandatory CKC partitions.

Operational completeness is evaluated against representative cases, declared obligations, and validators governed through Q and G; accountability concerns authority, rationale, lineage, and change control. A CKC can satisfy one dimension and fail the other, so conformance makes both observable.

Canonicality is an authority property, not a storage topology. K_c^v is canonical when it is the immutable contract authorized for capability c under declared conditions and selected by the Registry. It binds distributed governed sources through references and rules; assemblies and materializations remain derived projections.

Equation (3) distinguishes the living contract lineage from its immutable canonical versions:

$$\mathcal{L}_c(t) = \langle K_c^1 \prec K_c^2 \prec \cdots \prec K_c^{n_t} \rangle, \qquad \nu_t(c) = K_c^{a_t}, \quad 1 \le a_t \le n_t. \tag{3}$$

Here n_t is the latest governed version and a_t the active version. Evolution appends a successor with an explicit, decision-linked delta that may add, correct, supersede, deprecate, or remove commitments; earlier versions remain immutable. Atomic pointer activation applies the successor to later resolutions while preserving earlier assemblies.

At the architecture level, an *institutional capability lineage* is broader than CKC succession. Equation (4) models it as a Registry-addressable typed trace:

$$\Lambda_c(t) = \langle \mathcal{V}_c(t), \mathcal{R}_c(t) \rangle, \tag{4}$$

where $\mathcal{V}_c(t)$ contains capability c, exact CKC versions, assemblies, executions, evidence, decisions, and lifecycle records, and $\mathcal{R}_c(t)$ their typed links. Any retained record with a connected path to c belongs to the governed lineage, which the Registry indexes across canonical and execution state.

4.4 Capability Execution Environment

A *Capability Execution Environment* (CEE) is the situated boundary responsible for an admitted capability instance: a human or team, agent or multi-agent configuration, workflow, software service, or hybrid. Admission establishes a *bounded operational mandate* that delegates execution-scoped operational authority to the identified CEE; the assembly records its contractual limits and materialization delivers the corresponding usable representation to the local environment. The mandate authorizes situated execution, not institutional or canonical change. Within that mandate, the CEE controls reasoning, planning, working memory, local stores, tools, intermediate artifacts, and iterations, and may organize or distribute the delivery according to its own model. ICLA governs the institutional boundary, not internal cognition or coordination. Local knowledge remains under CEE or source authority; evidence is contract-selected rather than a wholesale transfer of working state. Each execution records the CEE identity, delivery configuration, intent, assembly, and submitted evidence. The CEE owns situated execution and submission; the Evidence Gateway verifies contractual conformity; governance decides institutional admission and activation.

4.5 Intent and resolution

An intent is $i_\eta = \langle g, x, \eta, u, r, b, q \rangle$, where g is the goal, x context, η the originating CEE, u its consumer and delivery configuration, r risk, b budget, and q required quality or assurance. It enters ICLA through an identified CEE boundary. Equation (5) maps it to registered capabilities:

$$\rho_t(i_\eta, \Gamma_t) \to \langle C_i, \omega_i, \pi_i \rangle, \tag{5}$$

where $C_i \subseteq \mathcal{C}_t$ is the selected capability set, ω_i its ranking and confidence, and π_i the rationale. Resolution snapshots the CEE, Registry state, and active CKCs; it may be rule-, graph-, model-based, or hybrid.

Candidate generation matches intent dimensions to Registry metadata. **Relation expansion** follows mandatory `depends_on` edges; other typed relations may affect ranking, inheritance, validation, or assembly planning. **Constraint validation** removes unauthorized, retired, conflicting, stale, or assurance-incompatible candidates and records why.

The resolver may compose several CKCs when no single capability is sufficient. Missing coverage, authority, freshness, or assurance yields a partial or inadmissible result; resolution MUST NOT invent a capability, contract, or authoritative assembly. Every admitted execution receives a new assembly record, including reused patterns.

4.6 Contextual assembly and materialization

Equation (6) derives a view from the intent and selected CKCs:

$$\Pi_{i,\eta} = \alpha\left(i_\eta, \{K_c^{v(c)} \mid c \in C_i\}, \mathcal{M}_t, P_t\right), \tag{6}$$

where P_t contains assembly and security policies. $\Pi_{i,\eta}$ records the CEE, Registry snapshot, exact CKCs, selections, exclusions, ordering, relations, procedures, provenance, and evidence and evaluation contracts. It is the immutable logical result of projecting canonical contracts and governed sources into situated execution knowledge. Materialization adapts it to substrate s as $B_{i,\eta}^s = \text{materialize}(\Pi_{i,\eta}, s)$; self-contained bundles, workspaces, messages, procedures, or governed access handles may retain the same contract and lineage trace. Their use within the admitted scope requires no new resolution; material changes to intent, coverage, authority, freshness, risk, or assurance do.

4.7 Execution evidence

Evidence from execution e is represented structurally as

$$O_{e,\eta} = \langle results, artifacts, M_e, incidents, decisions, lessons, provenance\rangle.$$

M_e reports governed metric identifiers, values, units, collection conditions, aggregation, and required uncertainty. Evidence traces to the CEE, intent, capabilities, exact CKCs, assembly, materialization, policies, and sources. It remains a candidate contribution until adjudication; undeclared measurements remain non-standard.

4.8 Adjudication and evolution

Let z_t denote an institutional maintenance event: a source-version change, qualified CEE evidence $O_{e,\eta}$, a policy revision, or an ownership or lifecycle event. Equation (7) evaluates its institutional consequences:

$$J_t(z_t, \Gamma_t, \{K_c\}, G_t) \to a_t, \tag{7}$$

where governance validates the event and, for execution evidence, its measurements against the applicable evaluation contracts. Adjudication may reject, quarantine, retain locally, refresh a binding or rule, create a successor CKC, revise Registry state, or propose a capability. Contract succession follows Equation (8):

$$K_c^{v+1} = \text{evolve}(K_c^v, z_t, a_t). \tag{8}$$

Only an approved decision may activate K_c^{v+1} in ν_{t+1}. Its record identifies the predecessor, accepted input, affected components, explicit delta, authority, validation, and rollback target. Resolution uses the current version until activation and the successor thereafter; retained assemblies preserve their original snapshot.

4.9 Capability crystallization as institutional discovery

For history H_t, Equation (9) detects recurrent patterns:

$$\chi(H_t) \to p_{c'}, \tag{9}$$

Table 3: Institutional capability relation types.

Relation	Meaning	Operational use
`depends_on`	Capability requires another capability	Impact analysis and mandatory assembly inclusion
`composes_with`	Capabilities are frequently used together	Candidate assembly templates and crystallization signals
`specializes`	Capability narrows another capability	Inheritance and override rules
`replaces`	Capability supersedes another	Migration, deprecation, and compatibility
`shares_knowledge`	CKCs reference overlapping sources	Change propagation and duplication control
`derived_from`	Capability originated from another or from a recurrent assembly	Crystallization lineage
`conflicts_with`	Simultaneous application needs resolution	Assembly validation and governance review

where $p_{c'}$ records the recurrent combination, stable assembly rules, value, candidate owner, overlap analysis, and draft CKC. Governed promotion follows Equation (10):

$$\text{promote}(p_{c'}) \to \langle c', K^1_{c'}, \Gamma_{t+1} \rangle. \tag{10}$$

Crystallization makes latent organizational structure reviewable: evidence may identify a candidate responsibility, but only governance assigns identity. The model now defines both the objects and their permitted transitions; the next section treats Registry navigation and lifecycle.

5 Registry Navigation and Lifecycle

The Registry turns the information model into an operational navigation surface and administrative control point. Resolvers query current capability state; authorized maintenance traces impacts, governs successors, changes lifecycle or ownership, and activates approved pointers. It stores identity, authority, relations, lifecycle, and bindings while institutional knowledge remains in governed source systems.

5.1 Registry relations

Table 3 defines the relation types used in resolution, assembly, impact analysis, and lifecycle management.

5.2 Institutional capability lifecycle

An institutional capability follows the lifecycle in Figure 3.

Capabilities may arise from strategy, process analysis, onboarding, or recurrent assemblies. Approval requires identity, ownership, outcome, an initial CKC, policy, and relation checks; retirement preserves lineage and redirects. Use may mature scope, ownership, relations, evaluation definitions, and assembly rules, while each transition remains governed and reversible where policy requires.

Registry navigation identifies responsible capabilities and active contracts. Execution then requires a strict separation between persistent authority, intent-specific assembly, and delivery-specific materialization.

6 Capability Contracts, Assemblies, and Persistence

CKCs are canonical contracts for knowledge and evaluation commitments; assemblies bind them to one admitted intent; materializations adapt an assembly to an execution substrate without acquiring contractual authority.

6.1 Persistence and authority rules

Table 4 defines persistence, mutability, and change authority for each information object.

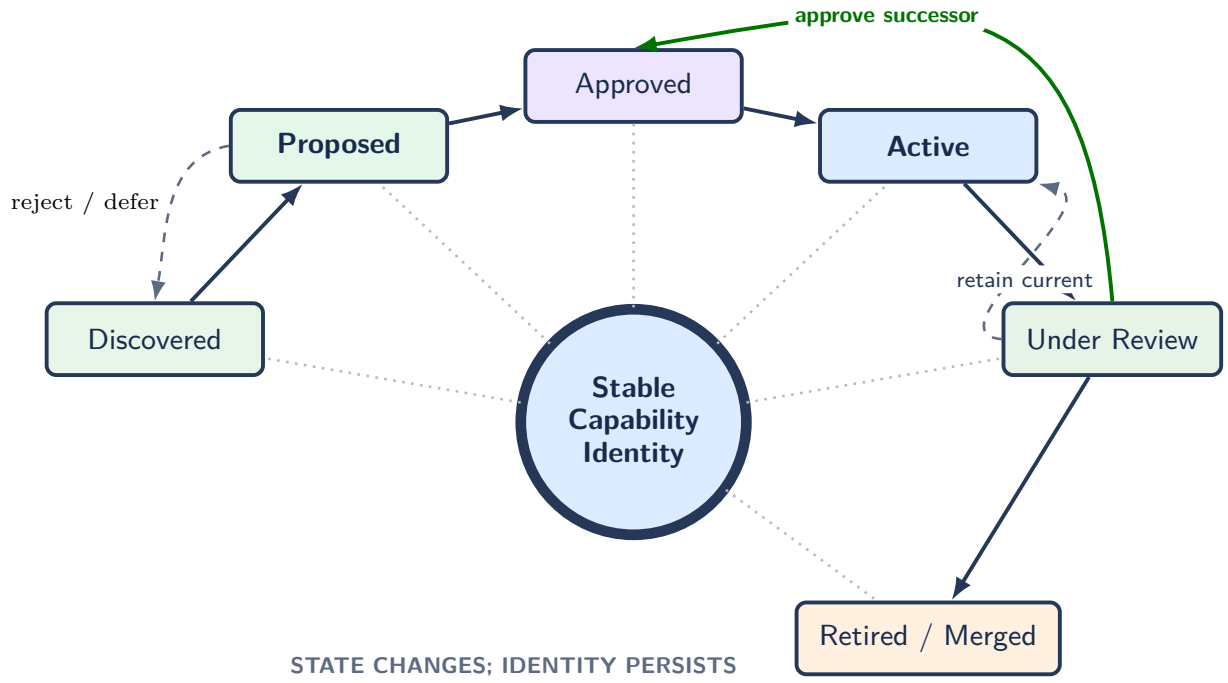

Figure 3: Lifecycle around a stable institutional identity. Capability state and active contracts may change, but the Registry-addressable identity persists through proposal, activation, review, succession, and retirement.

Table 4: Persistence, mutability, and authority rules.

Object	**Retention**	**Mutation**	**Authority and permitted transition**
Capability metadata and active CKC pointer	Persistent	Governed and versioned	Accountable owner proposes; authorized governance changes lifecycle, relations, or active pointer
CKC version / contract lineage	Persistent	Version immutable; successor-only	CKC authority proposes; governance approves a successor and its activation
Assembly definition / trace	Persistent metadata	Immutable per execution	Resolver and assembly service create it; no assembly may grant contractual authority
Physical materialization	Policy-dependent	Immutable	Delivery service creates a representation or governed access handles; neither may change assembly semantics
Execution-local memory	Local unless submitted	CEE-controlled	The CEE controls local state; submitted content enters the evidence path
Evidence bundle	Persistent	Append-only	Producer submits; Evidence Gateway validates conformity; governance determines disposition
Governance decision	Persistent	Append-only / superseding	Authorized reviewers record the decision and any resulting canonical transition
Source artifact	Source-policy dependent	Versioned	Source owner controls content; the Registry records bindings and impact paths

Materializations remain execution artifacts. The persistent assembly record retains identifiers, versions, selections, exclusions, hashes, timestamps, and evidence and evaluation contracts; payload retention follows policy.

6.2 Single- and multi-CKC assemblies

A simple intent may resolve to one CKC; another may require several active CKCs because no single registered capability covers the requested outcome. Equation (11) represents the selected institutional capability set:

$$C_i = \{c_1, c_2, \ldots, c_k\}, \quad k \geq 1. \tag{11}$$

For each admitted intent, the assembly engine selects, filters, orders, combines, and adapts active contracts and bound sources while preserving authority and provenance. A reused pattern still produces an intent-specific assembly record. If the selected set fails correctness or assurance conditions, the architecture returns failure or escalation rather than an authoritative delivery.

A recurring combination becomes an assembly pattern; promotion requires stable responsibility, distinct obligations, ownership, and demonstrated value.

6.3 Assembly correctness

For a selected capability set C_i, Equation (12) defines assembly validity as a conjunction:

$$\begin{aligned} \mathrm{Valid}(\Pi_{i,\eta}) = \; & \mathrm{Traceable} \wedge \mathrm{Authorized} \\ & \wedge \mathrm{RequiredCovered} \wedge \mathrm{EvaluationBound} \\ & \wedge \mathrm{ConflictsResolved} \wedge [\mathrm{Budget}(\Pi_{i,\eta}) \leq b]\,. \end{aligned} \tag{12}$$

These predicates require lineage, authorization, knowledge coverage, a bound evaluation contract, conflict resolution, and budget compliance. Here, RequiredCovered is evaluated against the representative cases, obligations, and validators declared in the active CKC evaluation contract. Validation may be deterministic, model-assisted, or human-reviewed according to risk.

7 Reference Architecture and Interaction Contracts

With information objects, lifecycle rules, and assembly conditions defined, the reference architecture assigns them to logical planes and exposes the operations that connect those planes.

Institutional Capability Lineages separates three logical planes. The **knowledge plane** holds distributed sources, evidence, CKC representations, and change signals. The **control plane** holds Registry identity, metadata, active pointers, impact analysis, resolution, policy, lineage, governance, and discovery state. The **distributed execution plane** contains heterogeneous CEEs that originate intents, receive materializations, reason and act with local resources, and return standardized evidence. CEEs may be independently located, operated, and secured; their coordination may be centralized, peer-to-peer, or hybrid. Orchestration and retrieval remain within the execution plane; Registry interaction resumes only for re-resolution or evidence submission. The Registry controls canonical capability state and its accountable evolution.

The architecture exposes logical operations for intent resolution, Registry navigation, assembly, materialization, evidence submission, adjudication, impact analysis, CKC activation, and capability proposal. Appendix A specifies their typed inputs, outputs, and responsibilities independently of concrete protocol.

An MCP implementation may expose resolution and assembly as tools, CKC and knowledge artifacts as resources, and governed procedures as prompts. Institutional authority remains in registry and governance records.

8 Governance, Evolution, and Institutional Discovery

The interaction contracts move work and evidence through the architecture. Governance determines whether those inputs remain local, extend an existing lineage, or justify a separate capability proposal.

8.1 Institutional maintenance and evidence path

Continuous maintenance forms an evidence-mediated institutional learning loop between distributed CEEs and canonical capability contracts. Source changes and CEE-produced knowledge enter a four-step governed

path before institutional state changes. Submitted execution outputs become episodic lineage records linked to identity, provenance, and applicable contracts. After qualification, conformity checking, and adjudication, they may remain historical, inform future projections, update governed source bindings, or motivate semantic, procedural, evaluation, or projection changes through an explicit successor CKC. Production or recurrence alone grants no institutional authority.

1. **Qualification**: Is the event relevant, sufficiently grounded, and complete enough for review?
2. **Conformity**: Do the change record, evidence, and measurements satisfy schemas, metric identifiers, units, collection rules, policies, provenance, and authorization?
3. **Adjudication**: Should the institution reject, retain, postpone, or promote the change?
4. **Propagation**: Which sources, CKCs, relations, caches, and future assemblies are affected?

Each stage records source, contract, metric, and policy versions; submitted values; impact paths; and decisions [21]. Detection may open a review automatically, but only governance can approve a successor or alter active Registry state.

8.2 Evolution outcomes

Figure 4 distinguishes adjudication within an existing capability lineage from the separate institutional process required to form a new capability. Rejection, quarantine, and local retention remain terminal dispositions within the first path.

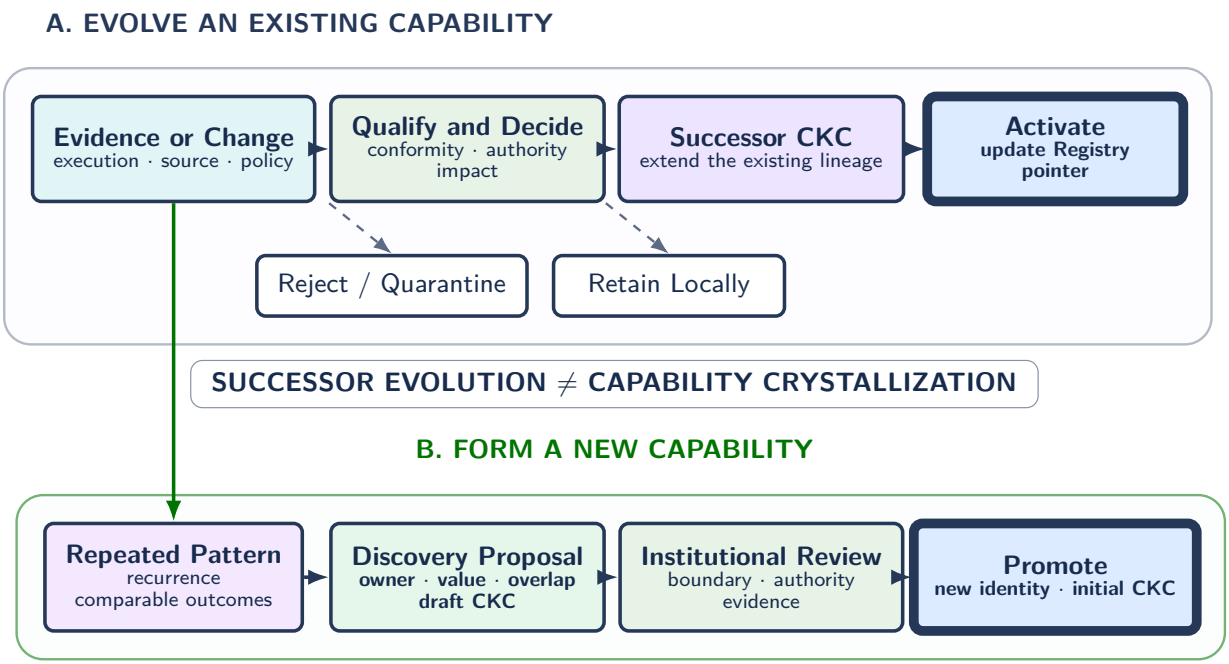

Figure 4: Two governed evolution paths. Successor evolution extends an existing CKC lineage; capability crystallization requires recurrent evidence, a discovery proposal, and separate review before the Registry assigns a new identity.

8.3 Crystallization criteria

Crystallization identifies stable responsibilities absent from the capability model. Beyond recurrence, a candidate requires:

- repeated demand across a declared time horizon;
- a stable responsibility distinct from existing institutional capabilities;
- a coherent and reusable assembly signature;

- measurable value or risk reduction under declared evaluation contracts;
- candidate ownership and governance authority;
- sufficient source and evidence coverage;
- manageable overlap and conflict with existing capabilities.

These criteria form a governed review gate rather than an automatic promotion score. Versioned policy may define thresholds or ranking heuristics, but no computed priority assigns institutional identity or authority.

9 Minimal Conformance Model

The preceding sections define architectural intent; the following invariants make ICLA behavior observable. *MUST*, *SHOULD*, and *MAY* follow RFC 2119 [35].

9.1 Mandatory invariants

ICLA-1 Institutional capability identity. Every active institutional capability MUST have a stable identifier, accountable owner, lifecycle status, and governed pointer to its active CKC version.

ICLA-2 Canonical capability contract. Every active institutional capability MUST reference an immutable CKC version within a governed lineage. The version MUST declare knowledge scope, operational relations, obligations, authorities, evidence schemas, source bindings, and projection rules. Its evaluation contract MUST define applicable metrics, units, collection conditions, thresholds, interpretation rules, and the representative-case or coverage basis used for completeness claims.

ICLA-3 CEE identity and governed authority. Every admitted execution MUST identify its CEE and consumer configuration. The CEE controls internal reasoning, working state, tools, and coordination; conformance MUST NOT require disclosure beyond the evidence contract. Its candidate contributions MUST enter through identified source or evidence paths and acquire institutional authority only through governed admission; no CEE MAY promote local state directly.

ICLA-4 Registry navigation. Institutional capabilities and active CKC pointers MUST be discoverable and filterable through metadata, lifecycle, policy, and conditions, and SHOULD be navigable through typed relations and source-impact paths.

ICLA-5 Intent and execution traceability. Every execution assembly MUST record the originating CEE, intent, Registry snapshot, and capability-resolution result. A resolution that lacks mandatory coverage, authority, freshness, or assurance MUST NOT produce an authoritative assembly.

ICLA-6 Assembly lineage. Every assembly MUST trace to exact CKC, evaluation-contract, source, policy, and transformation versions.

ICLA-7 Contract/projection separation. A CEE-specific materialization MUST NOT silently become a canonical CKC.

ICLA-8 Evidence and measurement separation. CEE outputs intended to influence institutional state MUST enter through an identified submission path and MUST NOT directly mutate the active CKC or other canonical state. Reported metrics MUST reference governed definitions or be marked non-standard and excluded from direct comparison and threshold-based decisions.

ICLA-9 Governed revision and activation. Source, policy, evidence, ownership, relation, or lifecycle changes that affect canonical state MUST create an impact record and governed decision. Contract changes MUST create a successor version; only an approved decision MAY update the active pointer used by future resolutions.

ICLA-10 Reproducibility. A retained assembly record MUST contain enough metadata to explain or reproduce the delivered view and interpret its measurement report, subject to retention and access policies.

ICLA-11 Institutional-discovery authority. Recurrent-use detection MAY create crystallization proposals, but only governed, traceable promotion MAY assign a new institutional capability identity in the Registry.

9.2 Conformance profiles

Table 5 groups the invariants into three cumulative profiles.

Table 5: Proposed ICLA conformance profiles.

Profile	Required capabilities	Suitable use
ICLA-Core	ICLA-1 to ICLA-7 and ICLA-10	Governed registry, distributed contribution, and repeatable delivery
ICLA-Governed	ICLA-Core plus ICLA-8 and ICLA-9	Regulated or high-accountability operational capabilities
ICLA-Evolving	ICLA-Governed plus ICLA-11, continuous impact analysis, candidate lifecycle, rollback	Longitudinal institutional learning and evidence-based capability discovery

Conformance depends on these behaviors and invariants across implementations.

10 Implementation Blueprint

Conformance constrains behavior across infrastructures. A practical deployment combines metadata and versioned object storage, existing organizational repositories, resolution and projection services, heterogeneous CEEs, an append-only Evidence Gateway, and governed review workflows. Appendix A gives a complete implementation mapping.

10.1 A minimal implementation sequence

A pilot can defer crystallization until execution history accumulates:

1. Create a small registry with stable identifiers, owners, lifecycle state, and three to five active CKCs with evaluation contracts.
2. Retain governed source knowledge in existing repositories; record bindings and ingest versioned source, policy, ownership, and lifecycle changes.
3. Add impact analysis and governed successor activation, then rule-based resolution, assembly of knowledge and metric definitions, and materializations for at least two heterogeneous CEEs, such as an agent runtime and a human review environment.
4. Record CEE and execution identity, reject inadmissible resolutions, validate standardized measurement reports, and govern explicit CKC successor deltas and active pointers.
5. Add pattern detection and capability-discovery workflows when sufficient history exists.

10.2 Companion Reference Specification and Repository

The companion repository operationalizes ICLA as a versioned reference kit. The paper defines the architecture, conformance invariants, and evaluation claims. The companion artifact specification is versioned as `icla-spec 0.1.0` and provides eight JSON Schema Draft 2020-12 contracts for institutional capability, CKC, Registry snapshot, intent, resolution and admission, contextual assembly, evidence bundle, and governance decision and activation. The paper's invariants define architectural conformance; schemas, traces, and executable artifacts provide inspectable realizations and do not introduce additional architectural requirements.

The artifact set is organized as follows:

- `schemas/`: machine-checkable contracts for the eight principal object types;
- `reference-objects/`: the consumer-independent object catalog and its schema mappings;
- `reference-traces/oauth-042/`: linked YAML records for the Registry snapshot, intent, resolution and admission, assembly, evidence bundle, governance decision, and activation;
- `reference-implementation/`: deterministic, schema-driven resolution, conformance validation, evidence qualification, adjudication, CKC activation, and lineage preservation;
- `tests/`: executable checks for end-to-end traces, conformance profiles, successor activation, historical preservation, and connected lineage.

The OAuth 2.1 trace exercises the lifecycle in Section 11. It resolves `INT-OAUTH-042` against `REG-SNAP-042`, snapshots six exact CKC versions in `ASM-OAUTH-042`, defines agent and reviewer materializations, returns five governed measurements and one explicitly non-standard measure, and records adjudication in `DEC-OAUTH-042`. Activation `ACT-VERIFY-010` makes `CKC-VERIFY v10` available only to future resolutions, while the retained assembly remains linked to v9; a later crystallization proposal requires separate review.

The schemas establish structural consistency; the reference traces establish end-to-end representational consistency; and the deterministic Reference Implementation and tests establish executable consistency across resolution, conformance, evidence qualification, governance, activation, historical preservation, and connected lineage. None of these artifacts establishes comparative effectiveness; Claims C1–C3 define the longitudinal evaluation still required.

GitHub: Institutional Capability Lineages

11 Worked Example: Governed Agentic Software Engineering

The blueprint becomes concrete in the companion's `reference-traces/oauth-042/` artifact set described in Section 10.2. Its linked YAML records serialize the Registry snapshot, intent, resolution and admission, assembly, evidence bundle, governance decision, and activation used below; the executable suite replays the same identifiers and checks resolution, conformance, successor activation, historical immutability, and connected lineage. A coding-agent runtime, deterministic validation services, and a human review environment are coordinated within composite CEE `CEE-OAUTH-042`. For trace simplicity, the three execution substrates share one CEE identity while retaining distinct materializations and local state. They consume the same institutional knowledge and evaluation contracts, then return artifacts and standardized measurements. Organizational Memory supplies requirements, decisions, policies, client inventory, code, tests, release procedures, incidents, and prior evidence.

The relevant Registry entries are `CAP-ARCH` (Architecture Governance), `CAP-IAM` (Identity and Access Architecture), `CAP-API` (API Evolution), `CAP-CHANGE` (Governed Code Change), `CAP-VERIFY` (Security and Compatibility Validation), and `CAP-RELEASE` (Controlled Release).

Before the intent arrives, governance approves `CKC-IAM v8` after an identity-policy change and activates it through `ACT-IAM-008`. Earlier assemblies remain linked to v7.

CEE `CEE-OAUTH-042` generates intent `INT-OAUTH-042`: *Introduce the OAuth 2.1 authorization protocol across a multi-service platform while preserving backward compatibility for approved clients, regulatory controls, and auditable release evidence.*

11.1 Registry resolution and admissibility

Resolution `RES-OAUTH-042` runs against Registry snapshot `REG-SNAP-042`. Table 6 records candidate generation, mandatory `depends_on` expansion, exclusions, and admission. `CAP-DATA` adds no relevant obligation, while existing validation and release contracts already cover the required telemetry from `CAP-OBS`. Authorization, freshness, assurance, and budget checks pass. The stricter compatibility rule resolves the only conflict, so `ADM-OAUTH-042` admits six capabilities.

Table 6: Concrete Registry-resolution and admissibility trace for `INT-OAUTH-042`.

Stage	**Concrete result**	**Persistent record**
Candidate generation	Direct: `CAP-IAM`, `CAP-API`, `CAP-CHANGE`, `CAP-VERIFY`; lower-ranked: `CAP-DATA`, `CAP-OBS`	`RES-OAUTH-042` with scores and rationale
Relation expansion	Add `CAP-ARCH` and `CAP-RELEASE` through mandatory `depends_on` edges	Registry snapshot and traversed edges
Filtering and conflict handling	Exclude data and observability candidates; apply authorization, freshness, assurance, and stricter compatibility rule	Validation trace with exclusions and policy versions
Final admission	Six capabilities retained; all mandatory assembly conditions satisfied	`ADM-OAUTH-042`: **admitted**

11.2 Assembly and materialization

The resolution snapshots six exact CKC versions: `ARCH v5`, `IAM v8`, `API v6`, `CHANGE v11`, `VERIFY v9`, and `RELEASE v4`. Assembly `ASM-OAUTH-042` selects the affected services, approved clients, contracts, code, tests, policies, incidents, and rollout procedures, while excluding credentials, unrelated data, superseded policies, and unaffected services. Functionally, it combines semantic commitments from policies, procedural guidance from migration and validation practices, and episodic knowledge from prior incidents and evidence; CKCs govern the bindings while sources remain distributed. The assembly binds metric definitions and produces agent workspace `MAT-AGENT-042` and human review packet `MAT-REVIEW-042`. Both are delivered through `CEE-OAUTH-042` with the same canonical trace, while their local state and reasoning remain separate. Figure 5 shows the path.

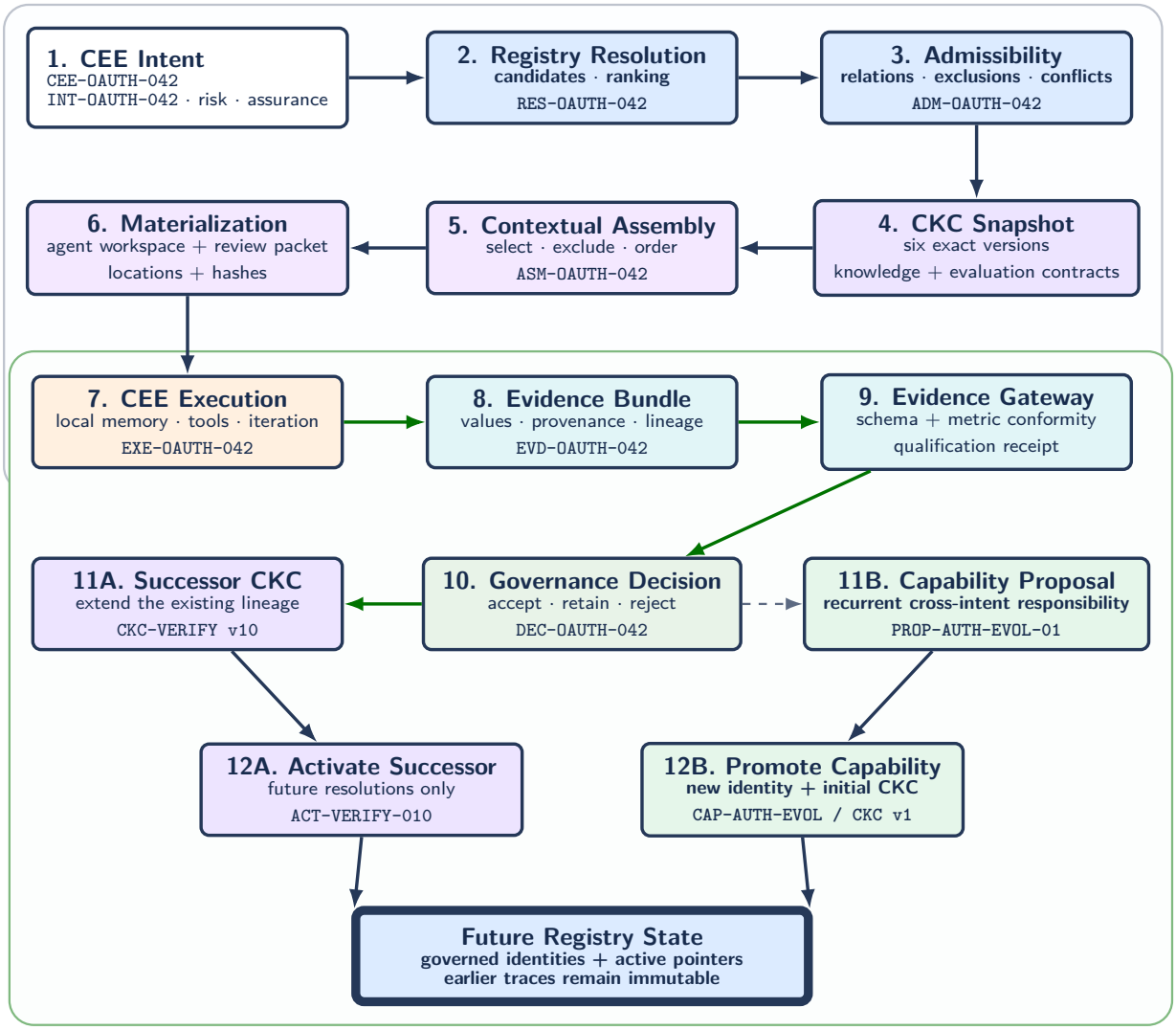

Figure 5: Auditable CEE trace. The environment receives exact CKC-derived materializations. Step 7 is an autonomous interval with local working state; selected evidence returns for qualification and adjudication while earlier traces remain immutable.

11.3 Execution, evidence, and adjudication

After `MAT-AGENT-042` and `MAT-REVIEW-042` are delivered, the composite CEE loads them into local working state. Agents may index, coordinate, invoke tools, and iterate; reviewers work independently, without continuous Registry, CKC, or assembly-service interaction. Only terminal or contract-defined checkpoint

outputs, measurements, decisions, and provenance enter evidence bundle `EVD-OAUTH-042`. It links them to the CEE, intent, Registry snapshot, admitted capabilities, assembly, exact CKC and source versions, materializations, and collection conditions. The resulting episodic record may remain precedent or support a procedural commitment in successor `CKC-VERIFY v10`.

Table 7 reports the measurements. All five governed metrics conform and pass. `agent.confidence` is retained as non-standard context but excluded from institutional comparison because no selected CKC defines it.

Table 7: Measurement report in evidence bundle `EVD-OAUTH-042`.

Metric identifier	Reported value	Governed threshold	Gateway result
`iam.profile.coverage`	100% endpoints	= 100%	Conforming; pass
`api.client.compatibility`	48/48 clients	= 100% approved clients	Conforming; pass
`verify.high.findings`	0 findings	= 0	Conforming; pass
`release.rollback.time`	4.2 minutes	$\leq$ 5 minutes	Conforming; pass
`verify.validation.latency`	34 minutes	$\leq$ 45 minutes	Conforming; pass
`agent.confidence`	0.91 score	No governed definition	Non-standard; excluded

Receipt `RCPT-OAUTH-042` qualifies the evidence for review. Decision `DEC-OAUTH-042` accepts the conforming artifacts and measurements, adds a reusable compatibility-test pattern to successor `CKC-VERIFY v10`, and retains a service-local migration exception as local evidence. `ACT-VERIFY-010` activates v10 only for future resolutions; `ASM-OAUTH-042` remains linked to v9.

11.4 Successor evolution and capability crystallization

The two evolution paths remain distinct. `DEC-OAUTH-042` extends `CAP-VERIFY`'s CKC lineage because the test pattern remains within its existing responsibility rather than defining a separate capability.

Later executions for token rotation, provider replacement, scope redesign, client migration, and protocol upgrades establish recurrence. The resulting proposal, `PROP-AUTH-EVOL-01`, includes a cross-intent assembly signature, comparable outcomes, candidate owner, overlap analysis, and draft CKC. Review may retain it as a template or promote `CAP-AUTH-EVOL` with `CKC-AUTH-EVOL v1`. Earlier traces remain unchanged.

The domain is software engineering; the architecture is domain-independent.

12 Evaluation Claims and Protocol

The worked trace demonstrates internal coherence; the following protocol states three falsifiable claims for comparative evaluation. In the ICLA condition, each run uses CKC-defined metrics, units, collection conditions, and thresholds, making outcomes comparable across CEEs and time. Sample sizes, directional hypotheses, non-inferiority margins, and acceptance thresholds are preregistered.

Claim C1—cross-CEE canonical stability. The unit is one recurring intent executed by heterogeneous CEEs—for example, an agent runtime, deterministic workflow, and human team—from the same governed sources and active CKCs. Measures are canonical object count, divergent authoritative copies, synchronization actions, CEE-specific exceptions, and task quality. C1 is supported when ICLA reduces divergence and synchronization against both baselines without violating declared quality or assurance margins.

Claim C2—continuous change accountability. The unit is one injected source, policy, obligation, relation, owner, or lifecycle change. Measures are affected-capability recall and precision, owner-identification time, review and activation latency, stale pointers, unexplained assembly changes, and reproducibility. C2 is supported by higher impact accuracy, fewer stale or unexplained states, and complete decision and activation traces.

Claim C3—controlled institutional discovery. The unit is one labeled recurrent or incidental responsibility pattern across execution history. Measures are proposal precision and recall, false promotion, reviewer agreement, owner-assignment time, duplication, rollback, and post-promotion value. C3 is supported when

preregistered precision and false-promotion limits are met and every promotion remains distinguishable from CKC succession and traceable to review evidence.

12.1 Comparative conditions

A longitudinal study should execute the same recurring workflow under three conditions:

1. **Retrieval baseline**: artifact retrieval without persistent institutional capability identity;
2. **Context-orchestration baseline**: governed context delivery without canonical institutional capability ownership or institutional discovery;
3. **ICLA condition**: registry-owned institutional capabilities, versioned CKCs, derived assemblies, governed evolution, and capability crystallization.

The study includes at least two heterogeneous CEEs, repeated intents, source and policy changes, rejected evidence, a successor CKC, and a recurrent assembly considered for discovery. Conditions share source snapshots, intent sequence, CEE configurations, and budgets. Order is counterbalanced; model and tool versions are pinned; and, where feasible, independent reviewers are blind to condition labels. Results report uncertainty and governance effort alongside task outcomes.

12.2 Measurement levels

Execution measures include task success, time, error, escalation, cost, and artifact quality. Internal measures cover Registry coverage, resolution accuracy, CKC scope, assembly relevance, policy compliance, provenance, and conflict detection. Institutional measures cover maintenance effort, evidence disposition, change-to-adjudication latency, divergence, stale-knowledge half-life, rollback, and Registry growth. Their definitions and collection conditions remain versioned in CKC evaluation contracts. Analysis separates boundary failures, projection failures, CEE execution failures, evidence-quality failures, and governance or evolution failures so that downstream performance is attributed to the responsible layer.

The registry and CKC layers are justified when their gains in reuse, impact analysis, and controlled evolution exceed their governance cost.

13 Scope and Limitations

This paper defines a reference architecture, conformance model, and worked trace. Its equations specify objects, relations, and governed transitions. The companion schemas, linked reference traces, deterministic Reference Implementation, and executable tests establish structural, representational, and executable consistency. They do not establish comparative effectiveness; Claims C1–C3 specify the longitudinal evaluation needed to assess it.

Capability boundaries depend on organizational purpose, ownership, risk, and language. ICLA is designed for recurrent, heterogeneous, auditable, volatile, or high-impact work whose continuity justifies ongoing governance. Evolution depends on observable events, reliable bindings, and timely adjudication; use alone may add noise or governance debt. The model assumes one logical Registry while allowing physically and organizationally distributed CEEs. Registry federation, cross-organization exchange, authority conflicts, privacy-preserving evidence, and formal capability equivalence remain open.

14 Conclusion

Institutional Capability Lineages (ICLA) governs the continuity of recurring capabilities across changing execution environments. It makes capability the stable institutional address that connects distributed governed sources, canonical contracts, situated execution, evidence, and controlled change.

The Registry provides navigable identity, ownership, relations, lifecycle, conditions, and active CKC pointers. Each CKC is an immutable, consumer-independent contract governing the capability-specific scope, authority, bindings, evaluation, and projection of semantic, procedural, and episodic organizational memory. Contextual assemblies translate those commitments into executable views for heterogeneous CEEs while preserving exact versions, selections, exclusions, and measures.

ICLA delegates execution, not institutional authority: authorized knowledge and obligations enter the CEE; contract-selected results return as evidence. Between those crossings, CEEs control local reasoning, working state, coordination, and intermediate knowledge. Adjudication may preserve submitted knowledge, update governed sources, or activate a successor CKC. Recurrent patterns follow a separate crystallization path toward a new identity, owner, and initial CKC. Both transitions preserve prior history.

ICLA joins what must persist with what must adapt. Capability identity and authority remain navigable and accountable; contracts, projections, relations, and evaluation definitions evolve through explicit evidence and decisions. The resulting lineage supports distributed reasoning and execution under unified, contract-governed, incremental, and auditable institutional learning.

A Logical Interfaces and Implementation Mapping

Table 8 defines the logical operations independently of concrete protocol.

Table 8: Logical interfaces.

Operation	Input	Output	Purpose
`resolve_intent`	CEE identity, intent, risk, budget	Ranked capabilities, rationale, confidence	Resolve a situated need before retrieving content
`get_capability`	Capability identifier, version policy	Metadata, active CKC, relations	Navigate and inspect the Registry
`assemble`	CEE intent, selected capabilities, policies	Assembly plan, trace, evaluation contract	Select knowledge and bind governed measures
`materialize`	Assembly identifier, CEE substrate	Bundle, payload, or access handles; metric schema, hash, expiry	Create an executable and measurable representation
`submit_evidence`	CEE and assembly identifiers, evidence report	Receipt, conformance, qualification status	Validate measures, preserve lineage, begin governance
`adjudicate`	Evidence identifiers, policies, reviewers	Decision record and resulting actions	Reject, retain, evolve, or propose
`propose_capability`	Pattern, rationale, owner candidate, draft CKC	Discovery-proposal identifier	Start crystallization before identity assignment
`impact_analysis`	Changed source, policy, CKC, relation, or capability	Affected CKCs, assemblies, CEEs, review case	Maintain freshness and identify governed work
`activate_ckc`	Capability, approved successor, decision record	Active-version update and propagation record	Publish a successor for future resolutions

Table 9 maps the logical architecture to one deployable configuration.

Table 9: One practical implementation mapping for ICLA.

Logical concern	**Candidate implementation**	**Stored information**
Registry and CKC management	Relational or graph metadata plus versioned object storage	Identity, ownership, relations, lifecycle, immutable CKCs, lineages, active pointers
Organizational Memory	Document, code, data, and knowledge repositories with change feeds	Versioned sources, bindings, retention, change events
Resolution and assembly	Rules, graph traversal, hybrid search, optional model assistance	Ranked capabilities, assembly plan, metric bindings, validation trace
Materialization and delivery	Workspace builder, serializer, REST, MCP, messaging, or files	Bundle, evaluation schema, URI, hash, expiry, execution handle
Distributed capability execution	Human workbench, agent runtime, workflow, software service, or hybrid	CEE identity and submitted outputs; local state remains under CEE authority
Evidence Gateway	Schema and metric validation with append-only storage	Results, artifacts, measurements, provenance, signatures
Governance and evolution	Review workflow, policy engine, audit log, dependency graph	Impact cases, decisions, successors, activation, rollback, proposals
Explorer interface	Registry APIs and graph queries	Search, navigation, versions, evidence, candidates

Acknowledgements

The author acknowledges the Laboratorio de Innovación Aplicada (L2IA) at Minsait (Indra Group) for fostering an environment that encourages scientific exploration in AI systems, software architecture, and organizational knowledge for intelligent systems.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[2] Zeyu Zhang et al. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024. doi:10.48550/arXiv.2404.13501.

[3] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025. doi:10.48550/arXiv.2507.13334.

[4] Haoyu Gao, Jai Lal Lulla, Hong Yi Lin, Sebastian Baltes, Christoph Treude, and Mansooreh Zahedi. From registry to repository: How AI agent skills are written, adapted, and maintained. *arXiv preprint arXiv:2607.00911*, 2026. doi:10.48550/arXiv.2607.00911.

[5] James P. Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991. doi:10.5465/amr.1991.4278992.

[6] Eric W. Stein. Organization memory: Review of concepts and recommendations for management. *International Journal of Information Management*, 15(1):17–32, 1995. doi:10.1016/0268-4012(94)00003-C.

[7] Maryam Alavi and Dorothy E. Leidner. Review: Knowledge management and knowledge management systems: Conceptual foundations and research issues. *MIS Quarterly*, 25(1):107–136, 2001. doi:10.2307/3250961.

[8] Jeanne W. Ross, Peter Weill, and David C. Robertson. *Enterprise Architecture as Strategy: Creating a Foundation for Business Execution*. Harvard Business School Press, Boston, MA, 2006.

[9] The Open Group. ArchiMate 3.2 specification. Technical report, The Open Group, 2022. URL https://www.opengroup.org/archimate-forum/archimate-overview.

[10] Endel Tulving. Episodic and semantic memory. In Endel Tulving and Wayne Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, New York, NY, 1972.

[11] Larry R. Squire. Declarative and nondeclarative memory: Multiple brain systems supporting learning and memory. *Journal of Cognitive Neuroscience*, 4(3):232–243, 1992. doi:10.1162/jocn.1992.4.3.232.

[12] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=BAakY1hNKS.

[13] Lijun Sun, Yijun Yang, Qiqi Duan, Yuhui Shi, Chao Lyu, Yu-Cheng Chang, Chin-Teng Lin, and Yang Shen. Multi-agent coordination across diverse applications: A survey. *arXiv preprint arXiv:2502.14743*, 2025. doi:10.48550/arXiv.2502.14743.

[14] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. AgentNet: Decentralized evolutionary coordination for LLM-based multi-agent systems. *arXiv preprint arXiv:2504.00587*, 2025. doi:10.48550/arXiv.2504.00587.

[15] Henry Peng Zou, Wei-Chieh Huang, Yaozu Wu, Yankai Chen, Chunyu Miao, Hoang Nguyen, Yue Zhou, Weizhi Zhang, Liancheng Fang, Langzhou He, Yangning Li, Yuwei Cao, Dongyuan Li, Renhe Jiang, and Philip S. Yu. A survey on large language model based human-agent systems. *arXiv preprint arXiv:2505.00753*, 2025. doi:10.48550/arXiv.2505.00753.

[16] Edwin Hutchins. *Cognition in the Wild*. MIT Press, Cambridge, MA, 1995.

[17] Robert M. Grant. Toward a knowledge-based theory of the firm. *Strategic Management Journal*, 17(S2):109–122, 1996. doi:10.1002/smj.4250171110.

[18] Pranav Subramaniam, Yintong Ma, Chi Li, Ipsita Mohanty, and Raul Castro Fernandez. Comprehensive and comprehensible data catalogs: The what, who, where, when, why, and how of metadata management. *arXiv preprint arXiv:2103.07532*, 2021. doi:10.48550/arXiv.2103.07532.

[19] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C Recommendation, World Wide Web Consortium, April 2013. URL https://www.w3.org/TR/prov-o/.

[20] ISO/IEC/IEEE. ISO/IEC/IEEE 42010:2022: Software, systems and enterprise—architecture description. Technical report, International Organization for Standardization, 2022. URL https://www.iso.org/standard/74393.html.

[21] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023. doi:10.6028/NIST.AI.100-1.

[22] Mariano Garralda-Barrio. Knowledge-centric information systems. *arXiv preprint arXiv:2607.02609*, 2026. doi:10.48550/arXiv.2607.02609.

[23] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, NY, 1995.

[24] Nicolai J. Foss, Kenneth Husted, and Snejina Michailova. Governing knowledge sharing in organizations: Levels of analysis, governance mechanisms, and research directions. *Journal of Management Studies*, 47(3):455–482, 2010. doi:10.1111/j.1467-6486.2009.00870.x.

[25] The Open Group. The TOGAF standard, 10th edition. Technical report, The Open Group, 2022. URL https://www.opengroup.org/togaf-standard.

[26] Pouya Aleatrati Khosroshahi, Matheus Hauder, Stefan Volkert, Florian Matthes, and Martin Gernegroß. Business capability maps: Current practices and use cases for enterprise architecture management. In *Proceedings of the 51st Hawaii International Conference on System Sciences*, pages 4603–4612, 2018. doi:10.24251/HICSS.2018.581.

[27] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025. doi:10.48550/arXiv.2510.04618.

[28] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025. doi:10.48550/arXiv.2506.06326.

[29] Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Chen, Jiawei Yang, Chunyu Li, Qingchen Yu, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025. doi:10.48550/arXiv.2507.03724.

[30] Gal Bakal. Knowledge activation: AI skills as the institutional knowledge primitive for agentic software development. *arXiv preprint arXiv:2603.14805*, 2026. doi:10.48550/arXiv.2603.14805.

[31] Federico Bottino, Carlo Ferrero, Nicholas Dosio, and Pierfrancesco Beneventano. Retrieval is not enough: Why organizational AI needs epistemic infrastructure. *arXiv preprint arXiv:2604.11759*, 2026. doi:10.48550/arXiv.2604.11759.

[32] Charafeddine Mouzouni. Context Kubernetes: Declarative orchestration of enterprise knowledge for agentic AI systems. *arXiv preprint arXiv:2604.11623*, 2026. doi:10.48550/arXiv.2604.11623.

[33] Google Cloud. Open Knowledge Format (OKF) v0.1 specification, 2026. URL https://github.com/GoogleCloudPlatform/knowledge-catalog/blob/main/okf/SPEC.md. Draft specification, accessed 2026-07-19.

[34] Model Context Protocol. Model Context Protocol specification, revision 2025-11-25. Technical report, Model Context Protocol Project, 2025. URL https://modelcontextprotocol.io/specification/2025-11-25. Accessed 2026-07-19.

[35] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, 1997. doi:10.17487/RFC2119.